

المحاضرة الخامسة: الجداول – القوائم Array - List

العنوان

1. الجداول أحادية البعد arrays
2. الجداول ثنائية البعد
3. فقط في بايثون
4. تمارين

المفاهيم الأساسية

- بنية الجداول arrays أهميتها وفائدة استخدامها.
- التعامل مع الجداول في بايثون باستخدام القوائم (أي بنية المعطيات list).

ملخص

مفهوم بنية المعطيات المركبة list (قائمة) في بايثون تقابل بنية الجداول arrays في لغات البرمجة الأخرى.

هذه البنية تسمح لنا كما في C-like بتعريف بنية المصفوفة متعددة الأبعاد، إذ أن المصفوفة ثنائية البعد هي جدول عناصره هي أيضاً جداول.

جميع الكلمات التالية هي مترادفات تعني نفس الشيء:

جدول، شعاع، مصفوفة أحادية البعد، قائمة في بايثون list.

أهداف تعليمية

يتعرف الطالب في هذا الفصل على:

- بنية المعطيات جدول/ مصفوفة array والمقابل لها في بايثون (بنية قائمة list).
- أهمية استخدام الجداول في التطبيقات البرمجية.
- أهمية القوائم في لغة بايثون كأحد عناصر القوة في كتابة التطبيقات بشكل موجز وفعال.

يمكن الطالب من:

كتابة برامج تستخدم بنية الجداول بشكل فعال في بناء تطبيقاته البرمجية.

1. الجداول أحادية البعد arrays

كي ندرك ضرورة استخدام الجداول (نسميها مصفوفات: أحادية البعد، ثنائية البعد، وتسميها بايثون قوائم list). سنبدأ بمثال تمهيدي.

1.1. ضرورة استخدام الجداول

سنأخذ مثال تمهيدي بسيط، وسنجد "صعوبة العيش" بلا جداول!

مثال: حساب نتائج انتخابية

لدينا 4 مرشحين لانتخابات، تصل نتائج الاقتراع على شكل دخل عددي، كل عدد على سطر يمثل رقم المرشح المُنتخب (صوت)، ننهي الدخل بعدد مميز وليكن الصفر.

نريد كتابة برنامج يسمح لنا بحساب أصوات المرشحين.

سنحاول حل المثال اعتماداً على بنى المعطيات البسيطة التي نعرفها حتى الآن.

لنتخيل أولاً بدقة ما نريد من البرنامج القيام به، بمثال عن دخله وخرجه:

الدخل سنستعوض هنا في الكتابة بـ ؛ بدلاً من نهاية السطر (توفيراً لعدد الأسطر):

2; 3; 3; 3; 4; 1; 1; 1; 3; 3; 2; 1; 2; 4; 4; 2; 1; 4; 4; 2; 2; 2; 2; 1; 0

الخرج:

المرشح 1: 6

المرشح 2: 8

المرشح 3: 5

المرشح 4: 5

```

#Variables initialization
C1 = C2 = C3 = C4 = 0
Se = input("vote for : ")
e = int(Se)
while( e != 0):
    # increment Ce
    if 1 <= e and e <= 4:
        if e == 1 : C1 += 1
        if e == 2 : C2 += 1
        if e == 3 : C3 += 1
        if e == 4 : C4 += 1
    Se = input("Voice for : ")
    e = int(Se)

#write results Ce
print("Candidate 1 = ", C1)
print("Candidate 2 = ", C2)
print("Candidate 3 = ", C3)
print("Candidate 4 = ", C4)

```

يعتبر الحل السابق بسيطاً واضحاً، ولكن لنفرض أننا أردنا زيادة عدد المرشحين إلى 10 مرشحين، سنضطر عندها لتعريف 10 متحولات بدل 4 ($C1, C2, \dots, C10$)، و 10 تعليمات شرطية if و 10 تعليمات كتابة، لتواكب الـ 10 مرشحين!

يمكن حل المسألة بإيجاز ووضوح عبر استخدام الجداول، حيث نعرّف جدولاً طوله يساوي عدد المرشحين.

إذن، بدلاً من تعريف عدة متحولات $C1, C2, \dots, C10$ سنعرّف جدول C طوله بعدد المرشحين. وسنخزن أصوات المرشح e في عنصر الجدول ذو الدليل e ، $C[e]$.

نفترض مبدئياً أن حجم الجدول كما في المثال السابق 4 مرشحين.

بالتالي ستحتسب الأصوات كما يلي:

كلما أدخلنا صوت e لأحد المرشحين نزيد واحد على قيمة عنصر المصفوفة $C[e]$.

```

NC=4 # Number of Candidates
#array initialization
C = [0]* (NC+1) # As C# int[] C=new int[NC+1] and fill it by
0

#Read e
Se = input("vote for : ")
e = int(Se)
while( e != 0):
    # increment Ce
    if 1 <= e and e <= 4:
        C[e] += 1
    # Read e
    Se = input("vote for : ")
    e = int(Se)
#write results C[]
for e in range(1,NC+1):
    print(" Candidate "+str(e)+"= "+str(C[e]))

```

استخدمنا الجدول C[] وحددنا عدد المرشحين في المتحول NC. يكفي فقط تغيير قيمة NC ليصبح برنامجنا قابل لحساب أصوات أي عدد من المرشحين!!! والبرنامج أقصر وأوضح، من قال أن الجداول/ المصفوفات هي للتطبيقات الرياضية الحسابية فقط!

2.1. ملاحظات عن الجداول/ القوائم في بايثون

- بنية المعطيات المكافئة للجداول في بايثون تدعى list سنسميها قائمة.
- في المثال: **type(C)** تعطي **list**.
- الأدلة مع القوائم تبدأ بالقيمة 0 كما في C-Like.
- الإشارة إلى عنصر في القائمة يستخدم [], كما في C-Like.
- لمعرفة طول القائمة نستخدم التابع **len()**.
- في المثال: **len(C)** تعطي 5.
- (في المثال حجزنا قائمة/ جدول طوله = عدد المرشحين + 1 : NC+1، كي يبقى عدد أصوات المرشح e في العنصر C[e] لا ننسى الأدلة تبدأ من 0).

إسناد قيم عناصر الجدول

- يمكن إسناد قيم لعناصر الجدول بسرد هذه القيم مباشرة ودون استخدام الأدلة.
- رأينا ذلك ممكناً في C-Like ولكن فقط في تعليمة التصريح عن المتحول الجدول.
- مثال ذلك وضع أسماء الأشهر في جدول:

- **في بايثون** يمكن إسناد قيم لعناصر الجدول بسرد هذه القيم مباشرة ودون استخدام الأدلة في أي موقع في البرنامج.

Python

```
month= ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
        "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
```

إن إسناد قيم لعناصر الجدول بسرد هذه القيم مباشرة **ودون استخدام الأدلة** مفيد في الإيجاز والوضوح في النص البرمجي.

مثال بسيط

معرفة اسم الشهر من رقم الشهر: هنا يجب استخدام 12 تعليمة شرطية. أما باستخدام الجداول ووضع أسماء الأشهر في جدول نحصل على الاسم مباشرة باستخدام الرقم كدليل إلى جدول الأسماء.

Python

```
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
          "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
m = input(" Month Number : ")
m = int(m)

# give month string abbreviation corresponding to number: m
print(" Month : ", m, " is ", months[m - 1]);
```

نسخ الجداول/ القوائم

- يجري نسخ الجداول بسهولة بالغة كما يلي:
- ```
Lf = ["abs", "round", "pow", "sorted"]
Lfcopy = Lf[:]
```
- ويمكن أيضاً استخدام التابع `copy()` لإجراء عملية النسخ مع ملاحظة أن النسختين تصبجان كل منهما مستقلتين أي إجراء تعديل على النسخة لا ينعكس على الأصل.

```
Lf = ["abs", "round", "pow", "sorted"]
we want to make a copy of months list
Lfcopy = Lf[:]
Lfcopy[0]= "cos"
print(f"Lf list is : {Lf}")
print(f"Lfcopy list is : {Lfcopy}")
```

```

Prints memory address in integer format
print(f"Address of Lf list in memory in integer
format is: {id(Lf)}")
Prints memory address in hexadecimal format
print(f"Address of Lf list in memory is:
{hex(id(Lf))}")

Prints memory address in integer format
print(f"Address of Lfcopy list in memory in
integer format is: {id(Lfcopy)}")
Prints memory address in hexadecimal format
print(f"Address of Lfcopy list in memory in
hexadecimal format is: {hex(id(Lfcopy))}")

thislist = ["apple", "banana", "cherry"]
mylist = thislist.copy() # make a copy of the
list
print(f"Address of thislist list in memory in
integer format is: {id(thislist)}")
print(f"Address of mylist list in memory in
hexadecimal format is: {id(mylist)}")

```

خرج الكود السابق:

Lf list is : ['abs', 'round', 'pow', 'sorted']

Lfcopy list is : ['cos', 'round', 'pow', 'sorted']

Address of Lf list in memory in integer format is: 2683562992640

Address of Lf list in memory is: 0x270d0d1a400

Address of Lfcopy list in memory in integer format is: 2683562844416

Address of Lfcopy list in memory in hexadecimal format is: 0x270d0cf6100

Address of thislist list in memory in integer format is: 2683564594560

Address of mylist list in memory in hexadecimal format is: 0x270d0efe980

## اختبار تساوي جدولين

رأينا في C-Like "خلف" الجداول هناك مؤشرات إلى الذاكرة التي تخزن عناصر الجدول. واختبار المساواة بين متحولين جدولين يقارن بين مؤشري الجدول. وليس بين القيم الفعلية للجدول.

- على العكس في بايثون تجري المقارنة على قيم عناصر الجدولين.
- $L1 == L2$  تعني تطابق القيم  $L1[i], L2[i]$  لجميع عناصر الجدولين.

```
lst1=[2,6,8]
lst2=[2,6,8]

print(lst1 == lst2)
for i in range(0,len(lst1)):
 print(i,lst1[i]==lst2[i])
```

## اختبار تساوي "عناوين" الجداول

ماذا لو أردنا اختبار تساوي مؤشرات/ عناوين جدولين على طريقة C-Like

يمكننا ذلك باستخدام الكلمة المفتاحية `is`:

```
lst1=[2,6,8]
lst2=[2,6,8]
lst3=lst2[:]
print(lst1 == lst3) # True
print(lst3 is lst2) # False
```

## المتتاليات الحسابية التابع `range()`

تعرفنا على التابع `range()` عندما شرحنا التعليمة التكرارية بعدد `for`.

```
for i in range(0,11):
```

تقوم التعليمة بتكرار التنفيذ من أجل قيم العداد  $i$  من قيمة 0 حتى 10.

- ما تقوم به `range()` هو توليد متتالية حسابية في حالتنا البداية 0 والنهاية 10 والخطوة 1.
- الشكل العام `range(s, e, d)` يولد القيم :

$s, s+d, s+2.d, s+3.d, ..$

$e$  مستثناة من القيم

إذن تحدد المتتالية الحسابية التي تولدها `range` بالقيم:  $s, e, d$  بداية، نهاية، خطوة.

- يكتسب التابع `range()` أهمية خاصة عند التعامل مع الجداول لأننا نستطيع مل قائمة بمتتالية من الأعداد محددة بالعناصر: بداية، نهاية، خطوة.

```
lst = list(range(0,25,2))
print(lst)
output [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24]
```

في المثال جرى تحويل القيم المولدة بـ `range()` (الأعداد الزوجية) وإسنادها إلى قائمة.

إعطاء قيم لعناصر القوائم/ الجداول باستخدام **for**

```
base2 = [2**i for i in range(0,11)]
print(base2)
output [1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024]
```

لاحظ الإيجاز والوضوح، فقط باستخدام التعليمة:

```
base2 = [2**i for i in range(0,11)]
```

جرى ملء الجدول base2 بقوى 2 من 0 إلى 10

القوائم في بايثون مثلها في "ماتلاب" **MatLab**

```
seq = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

شرائح من القائمة **slices**

يمكن أخذ قائمة جزئية من القائمة (شريحة) كالتالي:

```
seq = [0,1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
slice = seq[0:4]
print(slice)
```

ويكون الخرج:

```
[3 ,2 ,1 ,0]
```

يمثل البرنامج التالي الأساليب المختلفة لاقتطاع شرائح من القائمة:

```
seq = [0,1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

print(seq[0:4])
[0, 1, 2, 3]
print(seq[:4])
[0, 1, 2, 3]
print(seq[5:11])
[5, 6, 7, 8, 9, 10]
print(seq[5:])
[5, 6, 7, 8, 9, 10]
print(seq[0:10:2])
[0, 2, 4, 6, 8]
print(seq[:-5])
[0, 1, 2, 3, 4, 5]
```



## 2. الجداول ثنائية البعد

يمكن تعريف المصفوفات (الجداول ثنائية البعد) بأنها جدول عناصرها بدورها جداول. هكذا هو الأمر في C-Like وكذلك في بايثون هي قائمة كل عنصر فيه قائمة. في بايثون يُشار إلى أي عنصر في المصفوفة بدليلين السطر والعمود: C[line, colomn].

### 1.2. إنشاء وإسناد قيم

مثال تمهيدي: مثلث Pascal أو مثلث الكرخي

تأتي أهمية مثلث Pascal من استخداماته الكثيرة في العديد من التطبيقات الحسابية، فهو يستخدم مثلاً لحساب أمثال كثير الحدود  $(1+x)^n$ . إن نشر الصيغة  $(1+x)^n$  هو مجموع لحدود قوى  $x$  من القوة 0 حتى  $n$ ، أمثال هذه الحدود هي عناصر السطر  $n$  في مثلث Pascal. رياضياً تمثل قيمة العنصر بالعلاقة:

$$C_n^k = \frac{n!}{k!(n-k)!}$$

(تسمى توافيق  $n, k$  : عدد احتمالات اختيار  $k$  عنصر من مجموعة  $n$  بلا ترتيب). ويمكن إثبات العلاقة (\*).

$$C_n^k = C_{n-1}^k + C_{n-1}^{k-1}$$

ومن هذه العلاقة، كما سنرى، تأتي سهولة إنشاء مصفوفة Pascal. (مصفوفة مثلثية نصفها الأعلى مساوي 0).

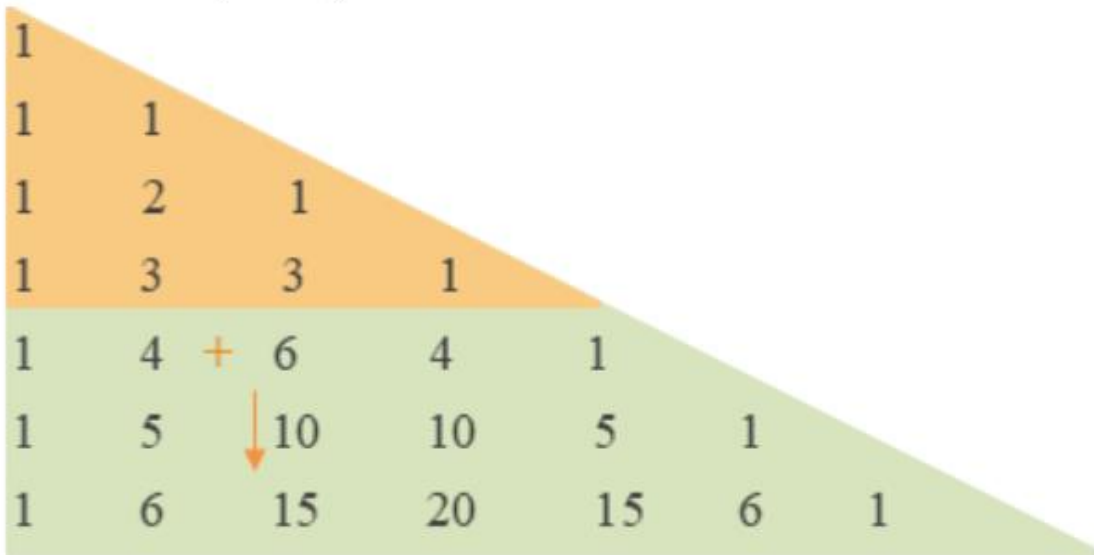
الفائدة: بعد ملء المصفوفة يمكن الاستغناء عن الحساب بالعلاقة  $n!, k!, \dots$  بالوصول إلى العنصر  $[n, k]$  في المصفوفة.

$$(1+x)^0 = 1$$

$$(1+x)^1 = 1 + 1 \cdot x$$

$$(1+x)^2 = 1 + 2 \cdot x + 1 \cdot x^2$$

$$(1+x)^3 = 1 + 3 \cdot x + 3 \cdot x^2 + 1 \cdot x^3$$



$$(1+x)^0 = 1$$

$$(1+x)^1 = 1 + 1 \cdot x$$

$$(1+x)^2 = 1 + 2 \cdot x + 1 \cdot x^2$$

$$(1+x)^3 = 1 + 3 \cdot x + 3 \cdot x^2 + 1 \cdot x^3$$

إن حساب القيم في هذا المثلث بسيط جداً، ولكنه سيفيدنا كمثال جيد عن التعامل مع المصفوفات ثنائية البعد، حيث سنقوم بخزن القيم في مصفوفة ثنائية البعد.

يمكن حساب قيم المثلث (عناصر المصفوفة) كما يلي:

**1.** إنشاء المصفوفة T: وسيكون لدينا أدلة الأسطر والأعمدة تبدأ من الصفر مما ينسجم مع حالة المسألة هنا. أبعاد المصفوفة هو  $n \times n$  (مثلاً  $n=6$  يعطي المثلث من المرتبة 5).

**2.** وضع أصفار في المصفوفة كاملة ما عدا العمود الأول فالقيم فيه تساوي 1 دوماً.

**3.** حساب القيم الأخرى بعلاقة بسيطة: (كما يظهر باللون الأحمر في الشكل السابق).

قيمة عنصر = مجموع قيم عنصرين من السطر السابق: عنصر العمود نفسه وعنصر العمود السابق. العلاقة (\*).

فإذا كان لدينا المصفوفة T، يحسب العنصر  $T_{i,j}$  من السطر i والعمود j وفق العلاقة أعلاه كما يلي:

$$T_{i,j} = T_{i-1,j} + T_{i-1,j-1}$$

```
n = 13 # Matrix Dimension

Matrix initialization
Fill with 0
T = [[0 for i in range(0,n)] for j in range(0,n)]
Fill 1 in first column
for i in range(0,n):
 T[i][0] = 1
Matrix Computation
for i in range(1,n):
 for j in range(1,n):
 T[i][j] = T[i-1][j-1] + T[i-1][j]
Matrix Printing
for i in range(0,n):
 row = ""
 for j in range(0,n):
 if T[i][j] < 10:
 row += " " + str(T[i][j])
 elif T[i][j] < 100:
 row += " " + str(T[i][j])
 else:
 row += " " + str(T[i][j])
 print(row)
```

ويكون خرج البرنامج كما يلي:

|   |    |    |     |     |     |     |     |     |     |    |    |   |
|---|----|----|-----|-----|-----|-----|-----|-----|-----|----|----|---|
| 1 | 0  | 0  | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0  | 0  | 0 |
| 1 | 1  | 0  | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0  | 0  | 0 |
| 1 | 2  | 1  | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0  | 0  | 0 |
| 1 | 3  | 3  | 1   | 0   | 0   | 0   | 0   | 0   | 0   | 0  | 0  | 0 |
| 1 | 4  | 6  | 4   | 1   | 0   | 0   | 0   | 0   | 0   | 0  | 0  | 0 |
| 1 | 5  | 10 | 10  | 5   | 1   | 0   | 0   | 0   | 0   | 0  | 0  | 0 |
| 1 | 6  | 15 | 20  | 15  | 6   | 1   | 0   | 0   | 0   | 0  | 0  | 0 |
| 1 | 7  | 21 | 35  | 35  | 21  | 7   | 1   | 0   | 0   | 0  | 0  | 0 |
| 1 | 8  | 28 | 56  | 70  | 56  | 28  | 8   | 1   | 0   | 0  | 0  | 0 |
| 1 | 9  | 36 | 84  | 126 | 126 | 84  | 36  | 9   | 1   | 0  | 0  | 0 |
| 1 | 10 | 45 | 120 | 210 | 252 | 210 | 120 | 45  | 10  | 1  | 0  | 0 |
| 1 | 11 | 55 | 165 | 330 | 462 | 462 | 330 | 165 | 55  | 11 | 1  | 0 |
| 1 | 12 | 66 | 220 | 495 | 792 | 924 | 792 | 495 | 220 | 66 | 12 | 1 |

## 2.2. مثال: ضرب مصفوفة بشعاع $B \cdot X = Y$

رياضياً، ضرب مصفوفة  $B$  بشعاع  $X$  يعطي شعاع  $Y$  عدد عناصره يساوي عدد أسطر المصفوفة  $B$ . شرط الضرب على أبعاد الجداول: إذا كان حجم  $B$  هو  $n \times m$  ( $n$  سطر و  $m$  عمود)، فيجب أن يكون طول الشعاع  $X$  مساوياً لـ  $m$  وسيكون طول الشعاع  $Y$  ناتج الضرب مساوياً لـ  $n$ . قيمة العنصر في سطر  $Y$  = مجموع جداء عناصر الشعاع  $X$  مع عناصر نفس السطر في المصفوفة  $B$ ، على الترتيب.

مثال: نرى في المثال التالي مصفوفة  $B$ ، شعاع  $X$ ، ناتج الضرب الشعاع  $Y = B \cdot X$

$$X = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 4 & 9 \end{bmatrix} \begin{bmatrix} 14 \\ 36 \end{bmatrix}$$

$$Y = B \cdot X \begin{bmatrix} 14 \\ 36 \end{bmatrix}$$

لاحظ حجم هذه الجداول: حجم  $B = 2 \times 3$ ، حجم  $X = 3$ ، حجم  $Y = 2$ .

لاحظ أيضاً: طريقة حساب عناصر الشعاع  $Y$ :

$$9 \times 3 + 4 \times 2 + 1 \times 1 = 36, \quad 3 \times 3 + 2 \times 2 + 1 \times 1 = 14$$

```
B = [[1, 2, 3],
 [1, 4, 9]]
X = [1, 2, 3]
n = len(B)
m = len(X)
Y = [0, 0] # len(Y) must be m = 2
for i in range(n):
 for j in range(m):
 Y[i] += B[i][j] * X[j]

print("B")
for i in range(0, len(B)):
 print(B[i])

print("\nX")
print(X)

print("\nY = B . X ")
print(Y)
```

ويكون الخرج

```
B
[1, 2, 3]
[1, 4, 9]

X
[1, 2, 3]

Y = B . X
[14, 36]
```

### 3. فقط في بايثون

#### 1.3. ملء جدول بقيم المتتالية الحسابية range()

كما سبق ذكره يمكن استخدام التابع range() لملء جدول بمجموعة قيم بعبارة برمجية موجزة وواضحة.

مثال ملء جداول أحادية البعد.

- ملء جدول بالأعداد الزوجية من 0 حتى N

```
N = 16
EvenA = [i for i in range(0, N + 1, 2)]
#EvenA = [0, 2, 4, 6, 8, 10, 12, 14, 16]
```

مثال ملء جداول ثنائية البعد

- ملء مصفوفة مثلث Pascal بقيم صفرية

```
T = [
 [0 for i in range(0,n)]
 for j in range(0,n)
]
```

- ملء مصفوفة بأرقام أدلتها (دليل السطر، دليل العمود) [i, j]

```
T = [
 ["[{0},{1}]".format(i,j) for i in range(0,n)]
 for j in range(0,n)
]
```

ستكون قيم T كالتالي

```
['[0,0]', '[1,0]', '[2,0]', '[3,0]', '[4,0]']
['[0,1]', '[1,1]', '[2,1]', '[3,1]', '[4,1]']
['[0,2]', '[1,2]', '[2,2]', '[3,2]', '[4,2]']
['[0,3]', '[1,3]', '[2,3]', '[3,3]', '[4,3]']
['[0,4]', '[1,4]', '[2,4]', '[3,4]', '[4,4]']
```

### 2.3. مقارنة مساواة قيم الجداول

في C-Like لا اختبار تساوي تطابق الجداول يجب كتابة تعليمة لمقارنة جميع عناصر الجدولين: وجدنا في أن اختبار تطابق قيم يجري بعملية == مباشرة على الجدولين. رائع!

### 3.3. جدول متعدد الأنماط

في المثال التالي سيحوي الجدول الأنماط **string, float, int**.

مثال: جدول أسماء الطلاب وعلاماتهم

سنقوم بكتابة برنامج يقوم بحساب محصلة علامة مادة وتحديد النجاح أو الرسوب. وإصدار نتائج النجاح والرسوب.

المعطيات موجودة في جدول يحتوي **أرقام وأسماء** الطلاب وعلامة **الامتحان** والأعمال في مادة. المعطيات موجودة في جدول وفق التنسيق التالي:

```
12436, Sami Ahmad, 80.8, 90.5
12440, Anas Mansour, 50.4, 80.5
13822, Moustafa Zein, 55.2, 70
```

سنظهر النتائج وفق الترويسة التالية:

```
Id, name, xam, pract, score
```

**Id** رقم الطالب، **name** الاسم، **xam** الامتحان، **pract** علامة الأعمال، **score** المحصلة وتُحسب وفق القاعدة:

70% امتحان 30% أعمال.

• يقوم البرنامج: بإظهار النتائج على الشاشة كما يلي:

```
12436, Sami Ahmad, 80.8, 90.5, 83.7, Succes
12440, Anas Mansour, 50.4, 80.5, 59.4, Fail
13822, Moustafa Zein, 39.0, 95, 55.8, Fail
```

تحدد نتيجة النجاح: وفق قواعد النجاح في المادة ضمن القيود التالية:

1. علامة الامتحان أكبر أو يساوي 40.
2. علامة الأعمال أكبر أو يساوي 40.
3. علامة المحصلة أكبر أو يساوي 60.

```
Id, name, xam, pract
Students = [[12436, "Sami Ahmad", 80.8, 90.5],
 [12440, "Anas Mansour", 50.4, 80.5],
 [13822, "Moustafa Zein", 57.2, 70],
 [12541, "Ahmad Alos", 39.4, 80.5],
 [16621, "Zouhair Dahrouj", 55.2, 35]]

for i in range(0, len(Students)):
 xam = Students[i][2]
 pract = Students[i][3]
 Score = xam * 0.70 + pract * 0.30
 Success = (xam>=40) and (pract >= 40) and (Score>=60)
 print(Students[i], ",%0.1f"%Score,",", "Success" if
Success else "Fail")
```

الخرج

```
[12436, 'Sami Ahmad', 80.8, 90.5] ,83.7 , Success
[12440, 'Anas Mansour', 50.4, 80.5] ,59.4 , Fail
[13822, 'Moustafa Zein', 57.2, 70] ,61.0 , Success
[12541, 'Ahmad Alos', 39.4, 80.5] ,51.7 , Fail
[16621, 'Zouhair Dahrouj', 55.2, 35] ,49.1 , Fail
```



## 4. تمارين

### 1.4. خوارزمية الفرز بالتعويم Bubble Sort

```
lstString = ["Zouhair", "Ahmad", "Zaid", "Mari", "Omar"]
print(lstString)
for i in range(0, len(lstString)):
 for j in range(i, len(lstString)):
 if (lstString[i]> lstString[j]):
 t = lstString[i]
 lstString[i] = lstString[j]
 lstString[j] = t
print("Sorted by bubble sort:")
print(lstString)
```

الخرج

```
['Zouhair', 'Ahmad', 'Zaid', 'Mari', 'Omar']
Sorted by bubble sort:
['Ahmad', 'Mari', 'Omar', 'Zaid', 'Zouhair']
```

يمكن التأكد من صحة النتيجة باستخدام التابع الجاهز **sorted()**.

اكتب التعليمة:

**sorted(lstString)**

قبل العمل بخوارزمية الفرز على الجدول **lstString**.

### الكلمات المحجوزة المستعملة

لا وجود لكلمات مفتاحية جديدة مع نمط القوائم في بايثون، فقط لنتذكر القوسين **[]**.  
ولكن أعطينا مثال استخدام **is** وهي ليست خاصة بالجدول بل لأي عنصرين من نمط مرگب.

القوائم Lists

list

```
names = ["Ahmed", "Nour", "Zied", "Sami"]
scores = [60, 84, 45, 100, 77, 93]
data = [7, "Python", 3.4, True]
print(data) [7, 'Python', 3.14, True]
```

القوائم Lists

list

1 | الفهرسة Indexing

```
names = ["Ahmed", "Nour", "Zied", "Sami", "Imen"]
```

| Index | 0  | 1  | 2  | 3  | 4  |                   |
|-------|----|----|----|----|----|-------------------|
|       |    |    |    |    |    | Positive indexing |
|       | -5 | -4 | -3 | -2 | -1 | Negative indexing |

```
names[0] # Get "Ahmed"
names[3] # Get "Sami"
names[7] # IndexError: list index out of range
```

names[-1] # Get "Imen"

names[-3] # Get "Zied"

القوائم Lists

list

2 | التقطيع Slicing

```
names = ["Ahmed", "Nour", "Zied", "Sami", "Imen"]
```

| Index | 0  | 1  | 2  | 3  | 4  |                   |
|-------|----|----|----|----|----|-------------------|
|       |    |    |    |    |    | Positive indexing |
|       | -5 | -4 | -3 | -2 | -1 | Negative indexing |

```
names[0:2] # Get ["Ahmed", "Nour"]
names[1:4] # Get ["Nour", "Zied", "Sami"]
names[:3] # Get ["Ahmed", "Nour", "Zied"]
names[2:] # Get ["Zied", "Sami", "Imen"]
```

```
names[:] # Get ["Ahmed", "Nour", "Zied", "Sami", "Imen"]
names[:2] # Get ["Ahmed", "Zied", "Imen"]
names[1:3] # Get ["Nour", "Imen"]
names[::-1] # Get ["Imen", "Sami", "Zied", "Nour", "Ahmed"]
```

# list[start:end:step]

نشاط:

(1) قم بتعريف متغير اسمه languages يحتوى على قائمة بأسماء اللغات البرمجية التالية:

PHP | Ruby | Swift | Go | C++ | JavaScript | Python

(2) إستخرج اللغات البرمجية Python, C++ و PHP من القائمة

(3) إستخرج أول ثلاث لغات برمجية من القائمة

(4) إستبدل اللغة البرمجية C++ بلغة C#

(5) إطبّع القائمة languages على الشاشة

كود برمجي مساعد لإنجاز النشاط السابق (جرب وناقش):

```
python lists.py > ...
1 languages = ["Python", "JavaScript", "C++", "Swift", "Ruby", "PHP"]
2
3 print(languages[0], languages[2], languages[-1])
4 print(languages[:3])
5 languages[2] = "C#"
6 print(languages)
```

لديك الكود البرمجي التالي (دقق ولاحظ الفرق وناقش):

```
1 l1 = ["Python"]
2 print(l1)
3 l2 = list("Python")
4 print(l2)
```

```
['Python']
['P', 'y', 't', 'h', 'o', 'n']
```

لاحظ الفرق؟

```
numbers = list(range(1, 20))
print(numbers)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
```

لاحظ البساطة؟

ماذا تفعل لتحصل على آخر رقم في القائمة 20؟

## نشاط:

- (1) قم بإنشاء list إسمها players تحتوي على الأسماء التالية:  
Omar | Wafa | Karim | Bilel
- (2) أحسب عدد اللاعبين داخل القائمة players
- (3) أضف Farah في نهاية القائمة players
- (4) إحذف Karim من القائمة players
- (5) قم بتغيير Wafa ب Ahmed في القائمة players
- (6) أوجد مكان Omar في القائمة players
- (7) قم بترتيب القائمة players ترتيبا تصاعديا
- (8) إطبّع القائمة players

الكود البرمجي للنشاط السابق (جرب لتتعلم):

```
players list.py > ...
1 players = ["Bilel", "Karim", "Wafa", "Omar"]
2 print(len(players))
3 players.append("Farah")
4 players.remove("Karim")
5 players[1] = "Ahmed"
6 print(players.index("Omar"))
7 players.sort()
8 print(players)
```

العوامل وبعض التوابع الجاهزة التي يمكن استخدامها مع القوائم (List):

| 1   Concatenate operator |                                                                                                |
|--------------------------|------------------------------------------------------------------------------------------------|
| Python Syntax            | Example                                                                                        |
| $+$                      | <pre>l1 = [1, 3, 5]<br/>l2 = [7, 9]<br/>l3 = l1 + l2<br/>print(l3) # Get [1, 3, 5, 7, 9]</pre> |



## 2 | Repetition operator

| Python Syntax | Example                                                                                                                               |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>*</b>      | <pre>l = ["Python", "C"] print(l * 2) # Get ["Python", "C", "Python", "C"] scores = [0] * 7 print(scores) # Get [0,0,0,0,0,0,0]</pre> |

## 3 | Membership operators

| Python Syntax | Example                                                                                                                    |
|---------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>in</b>     | <pre>fruits = ["Apple", "Banana", "Orange"] print("Orange" in fruits) # Get True print("Kiwi" in fruits) # Get False</pre> |
| <b>not in</b> | <pre>print("Orange" not in fruits) # Get False print("Kiwi" not in fruits) # Get True</pre>                                |

أمثلة على بعض التوابع التي يمكن استخدامها مع القوائم:  
إنشاء قائمة من الألوان:

```
colors = ["red", "green", "blue"]
```

|                                                                                                                  |                                                                        |
|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <pre># Add colors.append("yellow") colors.extend(["brown", "pink"]) colors.insert(0, "gray") print(colors)</pre> | <pre>['gray', 'red', 'green', 'blue', 'yellow', 'brown', 'pink']</pre> |
|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|

لاحظ الخرج وناقش؟

|                                                                                                                                                                                                                        |                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <pre># Remove # colors.pop(1) # colors.remove("red") del colors[0:2] colors.clear() print(colors)</pre>                                                                                                                | <p>جَرَبَ التّوابع في الكود البرمجي ولاحظ الفروقات بين هذه التّوابع التي كلها تُستخدم للحذف من القائمة.</p> |
| <pre>1 colors = ["red", "green", "blue", "red"] 2 3 # Find 4 if "orange" in colors : 5     print(colors.index("orange")) 6 print(colors.count("red"))</pre>                                                            | <p>جَرَبَ التّوابع في الكود البرمجي ولاحظ الفروقات بين هذه التّوابع التي كلها تُستخدم للبحث في القائمة.</p> |
| <pre>list functions.py &gt; ... 1 numbers = [9, 20, 3, 7] 2 3 # Other 4 numbers.reverse() 5 numbers.sort(reverse = True) 6 numbers_1 = numbers.copy() 7 numbers_1.append(30) 8 print(numbers) 9 print(numbers_1)</pre> | <p>جَرَبَ البرنامج التالي وناقش (من حيث خرج التّوابع المستخدمة)</p>                                         |
| <pre>list functions.py &gt; ... 1 numbers = [9, 20, 3, 7] 2 3 # General 4 print(len(numbers)) 5 print(min(numbers)) 6 print(max(numbers))</pre>                                                                        | <p>جَرَبَ البرنامج التالي وناقش (من حيث خرج التّوابع العامة المستخدمة)</p>                                  |
| <pre>nested lists.py &gt; ... 1 student = ["Mohamed", [75, 82.5, 90]] 2 student_mark = (student[1][0] + student[1][1] + student[1][2] * 2) / 4 3 print(student_mark)</pre>                                             | <p>استخدام القوائم المتداخلة (Nested Lists)</p>                                                             |

## المصفوفات (Array):



ملاحظة: المصفوفات عناصرها من نوع واحد فقط وهي تشغل حيزاً أقل من القوائم في الذاكرة.

**Syntax**

```
from array import array
array_name = array(typecode, [item1, item2, ...])
```

| Type code | Python type       |
|-----------|-------------------|
| "i"       | int               |
| "f"       | float             |
| "u"       | unicode character |

### نشاط:

- قم بإنشاء array إسمها scores تحتوي على القيم التالية:  
60 84 45 100 77 93
- قم بتغيير عدد نقاط (score) اللاعب الأول من 60 إلى 62
- أعرض score الخاص بأول ثلاثة لاعبين في المصفوفة
- أظف score 58 في آخر المصفوفة
- قم بحذف score اللاعب الثالث
- إطبع أكبر score في المصفوفة
- قم بترتيب المصفوفة scores ترتيباً تنازلياً
- إطبع المصفوفة scores

ملاحظة: إن تعليمة array تحتاج إلى النمط الذي سيتم ملئ المصفوفة به (typecode)

The actual representation of values is determined by the machine architecture (strictly speaking, by the C implementation). The actual size can be accessed through the [array.itemsize](#) attribute.

The module defines the following item:

#### `array.typecodes`

A string with all available type codes.

## array — Efficient arrays of numeric values

This module defines an object type which can compactly represent an array of basic values: characters, integers, floating-point numbers. Arrays are sequence types and behave very much like lists, except that the type of objects stored in them is constrained. The type is specified at object creation time by using a *type code*, which is a single character. The following type codes are defined:

| Type code | C Type             | Python Type       | Minimum size in bytes | Notes |
|-----------|--------------------|-------------------|-----------------------|-------|
| 'b'       | signed char        | int               | 1                     |       |
| 'B'       | unsigned char      | int               | 1                     |       |
| 'u'       | wchar_t            | Unicode character | 2                     | (1)   |
| 'w'       | Py_UCS4            | Unicode character | 4                     |       |
| 'h'       | signed short       | int               | 2                     |       |
| 'H'       | unsigned short     | int               | 2                     |       |
| 'i'       | signed int         | int               | 2                     |       |
| 'I'       | unsigned int       | int               | 2                     |       |
| 'l'       | signed long        | int               | 4                     |       |
| 'L'       | unsigned long      | int               | 4                     |       |
| 'q'       | signed long long   | int               | 8                     |       |
| 'Q'       | unsigned long long | int               | 8                     |       |
| 'f'       | float              | float             | 4                     |       |
| 'd'       | double             | float             | 8                     |       |

الكود البرمجي التالي يساعد في إنجاز النشاط السابق (جرب لتتعلم):

```
python arrays.py > ...
1 from array import array
2 scores = array("i", [60, 84, 45, 100, 77, 93])
3 scores[0] = 62
4 print(scores[0:3])
5 # Operators: +, *, in, not in
6 # Functions
7 scores.append(58)
8 scores.pop(2)
9 print(scores)
10 print(max(scores))
11 print(sorted(scores, reverse = True))
```



## التوابع مسبقة التعريف المستعملة

*chr()*, *float()*, *int()*, *len()*, *max()*, *open()*,  
*ord()*, *str()*

## التوابع المسبقة التعريف في بايثون Built-in functions (عددتها نحو 70)

|                      |                    |                     |                   |                       |
|----------------------|--------------------|---------------------|-------------------|-----------------------|
| <b>abs()</b>         | <b>dict()</b>      | <b>help()</b>       | <b>min()</b>      | <b>setattr()</b>      |
| <b>all()</b>         | <b>dir()</b>       | <b>hex()</b>        | <b>next()</b>     | <b>slice()</b>        |
| <b>any()</b>         | <b>divmod()</b>    | <b>id()</b>         | <b>object()</b>   | <b>sorted()</b>       |
| <b>ascii()</b>       | <b>enumerate()</b> | <i>input()</i>      | <b>oct()</b>      | <b>staticmethod()</b> |
| <b>bin()</b>         | <b>eval()</b>      | <i>int()</i>        | <i>open()</i>     | <i>str()</i>          |
| <b>bool()</b>        | <b>exec()</b>      | <b>isinstance()</b> | <i>ord()</i>      | <i>sum()</i>          |
| <b>bytearray()</b>   | <b>filter()</b>    | <b>issubclass()</b> | <b>pow()</b>      | <b>super()</b>        |
| <b>bytes()</b>       | <i>float()</i>     | <b>iter()</b>       | <i>print()</i>    | <b>tuple()</b>        |
| <b>callable()</b>    | <b>format()</b>    | <i>len()</i>        | <b>property()</b> | <i>type()</i>         |
| <i>chr()</i>         | <b>frozenset()</b> | <b>list()</b>       | <i>range()</i>    | <b>vars()</b>         |
| <b>classmethod()</b> | <b>getattr()</b>   | <b>locals()</b>     | <b>repr()</b>     | <b>zip()</b>          |
| <b>compile()</b>     | <b>globals()</b>   | <b>map()</b>        | <b>reversed()</b> | <b>__import__()</b>   |
| <b>complex()</b>     | <b>hasattr()</b>   | <i>max()</i>        | <b>round()</b>    |                       |
| <b>delattr()</b>     | <b>hash()</b>      | <b>memoryview()</b> | <b>set()</b>      |                       |

| Function             | Description                                                                                   |
|----------------------|-----------------------------------------------------------------------------------------------|
| <b>abs()</b>         | Returns the absolute value of a number                                                        |
| <b>all()</b>         | Returns True if all items in an iterable object are true                                      |
| <b>any()</b>         | Returns True if any item in an iterable object is true                                        |
| <b>ascii()</b>       | Returns a readable version of an object. Replaces none-ascii characters with escape character |
| <b>bin()</b>         | Returns the binary version of a number                                                        |
| <b>bool()</b>        | Returns the boolean value of the specified object                                             |
| <b>bytearray()</b>   | Returns an array of bytes                                                                     |
| <b>bytes()</b>       | Returns a bytes object                                                                        |
| <b>callable()</b>    | Returns True if the specified object is callable, otherwise False                             |
| <b>chr()</b>         | Returns a character from the specified Unicode code.                                          |
| <b>classmethod()</b> | Converts a method into a class method                                                         |
| <b>compile()</b>     | Returns the specified source as an object, ready to be executed                               |
| <b>complex()</b>     | Returns a complex number                                                                      |
| <b>delattr()</b>     | Deletes the specified attribute (property or method) from the specified object                |
| <b>dict()</b>        | Returns a dictionary (Array)                                                                  |
| <b>dir()</b>         | Returns a list of the specified object's properties and methods                               |
| <b>divmod()</b>      | Returns the quotient and the remainder when argument1 is divided by argument2                 |
| <b>enumerate()</b>   | Takes a collection (e.g. a tuple) and returns it as an enumerate object                       |
| <b>eval()</b>        | Evaluates and executes an expression                                                          |
| <b>exec()</b>        | Executes the specified code (or object)                                                       |
| <b>filter()</b>      | Use a filter function to exclude items in an iterable object                                  |
| <b>float()</b>       | Returns a floating point number                                                               |
| <b>format()</b>      | Formats a specified value                                                                     |

|                     |                                                                                    |
|---------------------|------------------------------------------------------------------------------------|
| <b>frozenset()</b>  | Returns a frozenset object                                                         |
| <b>getattr()</b>    | Returns the value of the specified attribute (property or method)                  |
| <b>globals()</b>    | Returns the current global symbol table as a dictionary                            |
| <b>hasattr()</b>    | Returns True if the specified object has the specified attribute (property/method) |
| <b>hash()</b>       | Returns the hash value of a specified object                                       |
| <b>help()</b>       | Executes the built-in help system                                                  |
| <b>hex()</b>        | Converts a number into a hexadecimal value                                         |
| <b>id()</b>         | Returns the id of an object                                                        |
| <b>input()</b>      | Allowing user input                                                                |
| <b>int()</b>        | Returns an integer number                                                          |
| <b>isinstance()</b> | Returns True if a specified object is an instance of a specified object            |
| <b>issubclass()</b> | Returns True if a specified class is a subclass of a specified object              |
| <b>iter()</b>       | Returns an iterator object                                                         |
| <b>len()</b>        | Returns the length of an object                                                    |
| <b>list()</b>       | Returns a list                                                                     |
| <b>locals()</b>     | Returns an updated dictionary of the current local symbol table                    |
| <b>map()</b>        | Returns the specified iterator with the specified function applied to each item    |
| <b>max()</b>        | Returns the largest item in an iterable                                            |
| <b>memoryview()</b> | Returns a memory view object                                                       |
| <b>min()</b>        | Returns the smallest item in an iterable                                           |
| <b>next()</b>       | Returns the next item in an iterable                                               |
| <b>object()</b>     | Returns a new object                                                               |
| <b>oct()</b>        | Converts a number into an octal                                                    |
| <b>open()</b>       | Opens a file and returns a file object                                             |

|                        |                                                                                 |
|------------------------|---------------------------------------------------------------------------------|
| <b>ord()</b>           | Convert an integer representing the Unicode of the specified character          |
| <b>pow()</b>           | Returns the value of x to the power of y                                        |
| <b>print()</b>         | Prints to the standard output device                                            |
| <b>property()</b>      | Gets, sets, deletes a property                                                  |
| <b>range()</b>         | Returns a sequence of numbers, starting from 0 and increments by 1 (by default) |
| <b>repr()</b>          | Returns a readable version of an object                                         |
| <b>reversed()</b>      | Returns a reversed iterator                                                     |
| <b>round()</b>         | Rounds a numbers                                                                |
| <b>set()</b>           | Returns a new set object                                                        |
| <b>setattr()</b>       | Sets an attribute (property/method) of an object                                |
| <b>slice()</b>         | Returns a slice object                                                          |
| <b>sorted()</b>        | Returns a sorted list                                                           |
| <b>@staticmethod()</b> | Converts a method into a static method                                          |
| <b>str()</b>           | Returns a string object                                                         |
| <b>sum()</b>           | Sums the items of an iterator                                                   |
| <b>super()</b>         | Returns an object that represents the parent class                              |
| <b>tuple()</b>         | Returns a tuple                                                                 |
| <b>type()</b>          | Returns the type of an object                                                   |
| <b>vars()</b>          | Returns the <code>__dict__</code> property of an object                         |
| <b>zip()</b>           | Returns an iterator, from two or more iterators                                 |